Software Engineering Lab

Lab 30

Leader Election dengan Fencing Tokens

Mencegah Split-Brain dan State Corruption Menggunakan Lease TTL dan Monotonic Revision



Lukman / lukman-ss

Masalah Nyata: Ilusi Otoritas Tunggal

Ketika node mengalami stop-the-world GC pause, koordinasi terdistribusi rentan terbelah.

UNANNOUNCED PAUSE

Poin 1

Garbage collector pause atau network hiccup menghentikan thread leader.

UNANNOUNCED PAUSE

Poin 2

Leader tidak dapat mengirim heartbeat renewal sebelum lease TTL habis.

UNANNOUNCED PAUSE

Poin 3

Node tidak menyadari bahwa ia sedang membeku dan kehilangan otoritas.

PREMATURE FAILOVER

Poin 1

Coordinator menganggap leader lama mati karena TTL kedaluwarsa.

PREMATURE FAILOVER

Poin 2

Kandidat berikutnya memenangkan election dan dipromosikan jadi leader baru.

PREMATURE FAILOVER

Poin 3

Dua node kini sama-sama menganggap dirinya pemegang mandat sah.

STALE WRITE HAZARD

Poin 1

Leader lama terbangun tanpa menyadari posisinya telah digantikan.

STALE WRITE HAZARD

Poin 2

Ia mengirim mutasi usang ke storage bersama menimpa data leader baru.

STALE WRITE HAZARD

Poin 3

Korupsi data senyap terjadi tanpa mekanisme validasi di storage.

Dalam arsitektur terdistribusi nyata, proses tidak pernah mati secara instan dan rapi. Fenomena runtime seperti stop-the-world garbage collection pause pada runtime modern atau network partition parsial dapat membekukan proses selama ratusan milidetik. Selama periode pause tersebut, lease TTL milik leader lama akan kedaluwarsa di coordinator, memicu failover otomatis ke kandidat baru. Ketika leader lama bangun dari pause, ia tidak menyadari waktu telah berlalu dan tetap mengeksekusi operasi tulis yang berpotensi merusak integritas data sistem.

Time passes invisibly to a paused process; local belief of leadership is never sufficient proof.

Mengapa Lease Berbasis Waktu Saja Gagal?

Pemeriksaan status lokal di sisi klien tidak memiliki jaminan linieritas saat eksekusi terdistribusi.

```
func (n *Node) DoWorkNaive(val string) error {
    if n.state != StateLeader { return ErrNotLeader }
    // GC pause terjadi tepat di sini selama 350ms!
    // Lease TTL 200ms kedaluwarsa di coordinator.
    // Node-B terpilih sebagai leader baru.
    return n.sharedStorage.Save(val) // Menimpa data Node-B!
}
```

Pendekatan naif mengasumsikan pengecekan status boolean lokal `if n.state == StateLeader` cukup menjamin mutual exclusion. Asumsi ini salah fatal dalam sistem asinkron karena adanya time-of-check to time-of-use (TOCTOU) gap. Tepat setelah pengecekan lokal berhasil, runtime thread dapat diinterupsi oleh GC pause atau penjadwalan OS selama ratusan milidetik. Saat thread kembali aktif, status kepemimpinan telah berpindah ke node lain, namun instruksi tulis tetap dikirim ke storage. Tanpa validasi di lapisan penyimpanan, write usang akan menimpa data terkini.

KEGAGALAN FUNDAMENTAL

Gap waktu antara pengecekan status (check) dan mutasi storage (act).
Asinkronitas jaringan dan runtime membuat pengecekan waktu lokal tidak dapat dipercaya.
Storage pasif menerima semua write tanpa memverifikasi urutan generasi pengirim.

Local validation suffers from TOCTOU gaps; true mutual exclusion must be verified by the shared target.

Mental Model: Otoritas Temporal vs Validasi Monotonik

Lease mengatur durasi giliran kerja, fencing token menegakkan urutan generations di storage.

```
type Lease struct {
    Key          string
    HolderID     string
    FencingToken int64         // revision monotonik naik
    ExpiresAt    time.Time
    TTL          time.Duration
}
```

Mental model arsitektur ini memisahkan dua konsep: hak memimpin sementara (lease) dan hak menulis permanen (fencing token). Lease coordinator mengelola kepemimpinan dengan TTL berbatas waktu, menerbitkan nomor tiket revisi yang strictly monotonically increasing setiap kali pemimpin baru dilantik. Penyimpanan bersama tidak lagi mempercayai identitas klien secara buta, melainkan bertindak sebagai gerbang independen yang hanya menerima mutasi jika tiket yang dibawa strictly lebih besar dari tiket terakhir yang tercatat.

PRINSIP FENCING TOKEN

Strict Monotonicity: Counter revision coordinator selalu bertambah di setiap kepemimpinan baru.
Resource Gatekeeping: Storage melacak lastSeenToken dan menolak token <= lastSeenToken.
Temporal Decoupling: Integritas data tidak bergantung pada sinkronisasi wall-clock mesin klien.
Self-Defending Store: Sistem kebal meskipun terjadi transient dual-leader state di memori.

Leases decide who should try to write; fencing tokens decide whose write is allowed to persist.

internal/coordinator/coordinator.go:15–28 • internal/storage/storage.go:14–25

Alur Koordinasi & Penolakan Mutasi Basi

Pipeline端到端: Campaign → Lease Grant → Normal Write → Pause/Failover → Stale Write Rejection.

SEKUENSI PIPELINE DISTRIBUSI

1. Node-A.Acquire() → Coordinator terbitkan Lease(Holder=A, Token=1, TTL=200ms)
2. Node-A.PerformFencedWrite(val) → Storage terima: incoming 1 > lastSeen 0 → lastSeen=1
3. Node-A mengalami Simulasi Pause 350ms (eksekusi berhenti, heartbeat terhenti)
4. Waktu berjalan melewati 200ms → Lease Node-A kedaluwarsa di Coordinator
5. Node-B.Acquire() → Coordinator terbitkan Lease(Holder=B, Token=2, TTL=200ms)
6. Node-B.PerformFencedWrite(val) → Storage terima: incoming 2 > lastSeen 1 → lastSeen=2
7. Node-A bangun dari pause → Mencoba Write dengan Token=1 usang
8. Storage Gatekeeper menolak: token 1 <= lastSeen 2 → ErrStaleFencingToken!

Pipeline koordinasi menunjukkan bagaimana kegagalan temporal ditangani secara deterministik. Tahap 1 hingga 2 memperlihatkan alur kerja normal di mana Node-A memenangkan election dan menulis sukses dengan token generasi 1. Pada tahap 3 hingga 5, Node-A mengalami pause melebihi batas TTL, memungkinkan coordinator melantik Node-B dengan token generasi 2 yang berhasil menulis ke storage. Saat Node-A tersadar di tahap 7 dan mencoba menulis dengan token 1, storage mengeksekusi fencing check dan menolak mutasi tersebut secara mutlak.

Coordinators manage failover transitions; storage gatekeepers enforce immutable monotonic ordering.

cmd/demo/main.go:15–77

Implementasi Gerbang Fencing di Shared Storage

Pemeriksaan atomik token <= lastSeenToken di bawah perlindungan mutex pada storage.go.

```
func (s *FencedStorage) Write(author string, token int64, value string) error {
    s.mu.Lock()
    defer s.mu.Unlock()

    if token <= s.lastSeenToken {
        return fmt.Errorf("%w: token %d <= last seen %d (author: %s)",
            ErrStaleFencingToken, token, s.lastSeenToken, author)
    }
    s.lastSeenToken = token
    s.records = append(s.records, Record{FencingToken: token, Author: author, Value: value, Timestamp: time.Now()})
    return nil
}
```

Potongan kode ini merupakan inti pertahanan terhadap split-brain di seluruh arsitektur sistem. Method Write pada struct FencedStorage mengunci mutex untuk menjamin operasi check-and-set berlangsung secara linier dan atomik. Aturan validasi mengevaluasi apakah token yang disertakan bernilai lebih kecil atau sama dengan lastSeenToken. Jika ya, storage langsung menolak operasi dengan mengembalikan ErrStaleFencingToken. Hanya token yang strictly lebih besar yang diizinkan memperbarui lastSeenToken dan mencatat data mutasi.

Strict inequality incoming > lastSeenToken turns disordered distributed writes into a monotonic stream.

internal/storage/storage.go:33-49

Failure Path: Penolakan Stale Write Pascajeda GC

Log terminal membuktikan Node-A ditolak setelah Node-B mengambil alih kepemimpinan.

TERMINAL FAILURE LOG

```
[4] Simulating Stop-the-World GC Pause / Network Partition on Node-A
(350ms > TTL 200ms)...
[5] Status after TTL expiration: Node-A=LEADER, Node-B=LEADER
[6] New Leader promoted: Node-B (Fencing Token: 2)
[7] New Leader Node-B performing fenced write...
Write SUCCESS by Node-B (Token 2)
[8] Old Leader Node-A wakes up from GC pause (still holding stale token 1)...
[9] Old Leader Node-A attempts write to shared storage using stale token 1...
Write REJECTED by FencedStorage: stale fencing token: rejected to prevent
split-brain write: token 1 <= last seen 2 (author: Node-A)
```

ANALISIS KEGAGALAN

Pada baris [5], terjadi transient dual-leader: Node-A dan Node-B sama-sama berstatus LEADER.
Meskipun Node-A merasa sah, storage berhasil mendeteksi anomali generasi (token 1 <= 2).
Shared storage history tetap bersih: hanya mencatat record dari generasi kepemimpinan yang valid.

Failure path konkret ini diambil langsung dari hasil eksekusi program demo lab. Pada langkah 4 hingga 5, Node-A dipause selama 350 milidetik, melampaui batas TTL 200 milidetik sehingga lease kedaluwarsa. Menariknya, sistem sempat berada pada kondisi transient dual-leader di mana kedua node menganggap dirinya sebagai LEADER aktif. Namun, saat Node-A terbangun dan mencoba menulis dengan token 1, lapisan penyimpanan menolak operasi tersebut secara seketika karena token 1 lebih kecil dari token 2 milik Node-B.

Transient dual-leader states are inevitable in asynchronous networks; fencing tokens make them safe.

cmd/demo/main.go:61–75 • engineering/03-execution-result.md:61–70

Mekanisme vs Gejala: Evaluasi Solusi Koordinasi

Mengapa solusi parsial tetap menyisakan celah korupsi data pada kegagalan terdistribusi.

Pendekatan	Mekanisme Koordinasi	Gejala / Failure Mode	Garansi Split-Brain
Heartbeat Naif	Ping status via interval	Network partition memicu dual-leader	Tidak Aman (korupsi data)
Lease TTL Murni	Kunci kedaluwarsa otomatis	GC pause memicu stale write pasca-wake	Tidak Aman (TOCTOU gap)
Distributed Lock	Mutex terpusat (Redis SETNX)	Lock lepas sebelum job tuntas	Sebagian (hanya efisiensi)
Lease + Fencing	Monotonic revision + check-and-set	Stale write dihentikan di resource	Aman Mutlak (linierizable)

Tabel ini membandingkan empat pendekatan koordinasi terdistribusi dalam menghadapi kegagalan jaringan dan runtime pause. Heartbeat naif dan lease TTL murni gagal melindungi integritas data karena keduanya rentan terhadap jeda eksekusi dan kondisi split-brain. Penggunaan distributed lock tanpa token urutan hanya cocok untuk optimalisasi efisiensi seperti deduplikasi job. Hanya penggabungan antara lease berbatas waktu dengan fencing token monotonik yang memberikan garansi keamanan mutlak pada level resource.

Locks optimize efficiency; monotonic fencing tokens protect data correctness.

3 Keputusan Desain Kritis Leader Election

Urutan keputusan arsitektur untuk membangun koordinasi terdistribusi yang tangguh.

1. MONOTONIC CLOCK FOR LEASES

Wajib gunakan time.Now() (monotonic reading) untuk evaluasi TTL dan renewal.
Mencegah anomali perpindahan waktu akibat sinkronisasi NTP step clock drift.
Terapkan interval renewal jauh lebih cepat dari TTL (contoh: renewRate <= TTL / 3).

2. STRICT MONOTONIC REVISION

Coordinator wajib mengincrement counter revision secara atomik di setiap lease baru.
Hindari penggunaan UUID atau hash acak karena tidak memiliki relasi urutan (ordering).
Renewal pada holder yang sama mempertahankan nilai token yang sudah ada.

3. ATOMIC CHECK-AND-SET GATE

Storage wajib menegakkan aturan incomingToken > lastSeenToken di bawah mutex/CAS.
Tolak operasi dengan status kode kesalahan eksplisit (ErrStaleFencingToken).
Simpan histori watermark generasi tertinggi yang pernah berhasil diterima.

Implementasi koordinasi yang aman bergantung pada tiga keputusan desain fundamental. Pertama, perhitungan masa berlaku lease harus menggunakan monotonic clock untuk menghindari disrupsi penyesuaian jam sistem via NTP. Kedua, setiap pergantian kepemimpinan wajib menerbitkan revision token yang monotonik naik; UUID acak dilarang karena tidak memiliki relasi urutan kausal. Ketiga, gerbang validasi di lapisan penyimpanan bersama harus mengimplementasikan check-and-set atomik yang menolak token yang tidak lebih besar.

Monotonic clocks measure duration; monotonic counters establish immutable authority sequence.

Bukti Eksekusi: Siklus Lengkap Election & Fencing

Output langsung dari go run ./cmd/demo memperlihatkan transisi kepemimpinan dan integritas storage.

DEMO EXECUTION TRACE

```
=== Distributed Leader Election with Fencing Tokens Demo ===
[1] Starting Node-A and Node-B election campaigns...
Status: Node-A=LEADER, Node-B=FOLLOWER
[2] Leader elected: Node-A (Fencing Token: 1)
[3] Node-A performing legitimate fenced write... Write SUCCESS by Node-A (Token 1)
[4] Simulating Stop-the-World GC Pause / Network Partition on Node-A (350ms > TTL 200ms)...
[5] Status after TTL expiration: Node-A=LEADER, Node-B=LEADER
[6] New Leader promoted: Node-B (Fencing Token: 2)
[7] New Leader Node-B performing fenced write... Write SUCCESS by Node-B (Token 2)
[8] Old Leader Node-A wakes up from GC pause (still holding stale token 1)...
[9] Old Leader Node-A attempts write to shared storage using stale token 1...
Write REJECTED by FencedStorage: stale fencing token: rejected to prevent split-brain write: token 1 <= last seen 2 (author: Node-A)
Result: Split-brain prevented! Old leader's stale write was safely rejected.
[10] Shared Storage History:
1. Token: 1 | Author: Node-A | Value: record-from-initial-leader
2. Token: 2 | Author: Node-B | Value: record-from-failover-leader
```

Output terminal ini membuktikan bahwa seluruh siklus kepemimpinan berjalan sesuai spesifikasi desain. Node-A pertama kali dilantik dengan Token 1 dan berhasil menulis record pertama. Saat Node-A dibekukan selama 350 milidetik, Node-B mengambil alih kepemimpinan dengan Token 2 dan menulis record kedua. Ketika Node-A terbangun dan berusaha menulis dengan Token 1, sistem menolak operasi tersebut secara eksplisit. Riwayat penyimpanan bersama membuktikan hanya ada dua entri sah dan tidak ada data yang tertimpa oleh mutasi usang.

Terminal logs provide indisputable evidence: stale writes fail loudly, preserving storage state.

YANG TIDAK DIBUKTIKAN

Batasan ruang lingkup laboratorium yang wajib dipahami sebelum implementasi produksi.

YANG TIDAK DIBUKTIKAN

Coordinator bersifat single-node in-memory; tidak membuktikan konsensus multi-node Raft riil.
Tidak menguji fault injeksi disk fisik, network loss paket parsial, atau Byzantine failures.
Clock offset diuji melalui manipulasi software; tidak menguji hardware clock skew ekstrem.
Storage diimplementasikan via in-memory sync.RWMutex; bukan PostgreSQL CAS atau etcd revision check.
Nilai waktu (TTL 200ms, renew 50ms) bersifat ilustratif lab; bukan panduan tuning produksi.
Tidak mencakup mekanisme automated fencing token compaction atau database sequence recycling.

Klaim teknis dalam laboratorium ini dibatasi oleh parameter lingkungan pengujian yang disederhanakan. Coordinator yang digunakan berjalan in-memory di dalam satu proses Go, bukan kluster terdistribusi multi-node berbasis konsensus Raft seperti etcd atau Paxos. Uji coba tidak mensimulasikan kegagalan jaringan fisik nyata seperti packet loss berkala, korupsi hardware disk, atau perilaku Byzantine. Nilai parameter waktu seperti TTL 200 milidetik hanya dirancang untuk kecepatan eksekusi demo dan tidak boleh langsung diaplikasikan ke lingkungan produksi tanpa benchmarking p99 latency.

Acknowledge lab constraints to avoid confusing conceptual proofs with production-grade topology.

Trade-off Arsitektur: Fencing vs Kompleksitas

Menilai biaya operasional penambahan gerbang fencing dibandingkan manfaat konsistensi data.

MANFAAT & KEUNGGULAN

Jaminan Konsistensi Mutlak: Menghilangkan risiko split-brain write pada shared storage.
Decoupled Failure Recovery: Node baru dapat langsung menulis tanpa menunggu node lama mengonfirmasi mati.
Auditability: Setiap record memiliki fencing token yang merefleksikan generasi pemimpin penulis.

KONSEKUENSI & BIAYA

Modifikasi Storage Layer: Resource bersama wajib mendukung skema token dan check-and-set.
Failover Latency: Masa transisi pemimpin baru terikat pada durasi TTL lease yang dipilih.
Client Error Handling: Aplikasi klien wajib memiliki alur penanganan saat write ditolak ErrStaleFencingToken.

Penerapan fencing tokens menuntut kompromi arsitektural antara keamanan data dan kompleksitas sistem. Keunggulan utamanya adalah eliminasi total terhadap korupsi data akibat split-brain serta kemampuan failover tanpa perlu menunggu konfirmasi kematian node lama. Namun, biaya penerapannya tidak kecil: lapisan penyimpanan harus dimodifikasi agar mampu mengevaluasi token, waktu failover dibatasi oleh durasi TTL lease, dan klien aplikasi harus memiliki logika fallback ketika transaksi ditolak akibat token kedaluwarsa.

Safety requires storage collaboration; you trade write simplicity for bulletproof consistency.

Validasi Konkurensi: 5 Node Berlomba Rebut Lease

TestConcurrentElectionRace membuktikan linearitas coordinator: tepat 1 leader terpilih.

```
func TestConcurrentElectionRace(t *testing.T) {
    coord := coordinator.New()
    store := storage.New()
    nodes := make([]*candidate.Node, 5)
    for i := 0; i < 5; i++ {
        nodes[i] = candidate.NewNode(string(rune('A'+i)), "leader-lock", 100*time.Millisecond, 20*time.Millisecond, coord, store)
        nodes[i].Start(ctx)
    }
    time.Sleep(100 * time.Millisecond)
    // Assert: leaderCount == 1 di antara 5 node yang berlomba
}
```

HASIL VERIFIKASI

Race Detector: PASS (bebas dari data race condition).
Linearizable Mutex: Semua permintaan Acquire dan Renew dieksekusi secara serial.
Tepat 1 Leader: 4 kandidat lainnya tetap berstatus FOLLOWER secara stabil.

Uji konkurensi membuktikan ketahanan implementasi saat lima node kandidat mencoba mengakuisisi lease secara simultan. Coordinator memanfaatkan sync.Mutex internal untuk memastikan setiap pengecekan dan penetapan lease berjalan secara linearizable. Dalam pengujian TestConcurrentElectionRace, setelah 100 milidetik kompetisi intensif, tepat satu node berhasil dipromosikan menjadi LEADER sementara empat node lainnya tertahan sebagai FOLLOWER. Verifikasi dengan flag -race membuktikan tidak ada data ri proses.

Serialized coordinator primitives resolve concurrent contention into a single unambiguous leader.

tests/election_test.go:156–191

Before vs After: Evolusi Sistem Koordinasi

Transformasi dari koordinasi rapuh berbasis waktu lokal menuju arsitektur fenced mutlak.

BEFORE (LEASE TTL MURNI)

Koordinasi: Node mengandalkan local state check sebelum eksekusi.
Failure Mode: Stop-the-world pause memicu penulisan ganda pasca-wake.
Penyimpanan: Storage pasif menerima semua data tanpa validasi urutan.
Dampak: Korupsi data senyap dan state database tidak konsisten.

AFTER (LEASE + FENCING TOKENS)

Koordinasi: Lease berbatas waktu dipadukan dengan revision counter monotonik.
Failure Mode: Leader lama terbangun, namun mutasinya otomatis ditolak.
Penyimpanan: Storage aktif memeriksa aturan incomingToken > lastSeenToken.
Dampak: Integritas data terjamin 100% aman dari ancaman split-brain.

Perbandingan before-after ini menunjukkan pergeseran paradigma dalam menangani koordinasi terdistribusi. Sebelum menerapkan fencing tokens, sistem sangat rapuh terhadap jeda eksekusi karena penyimpanan mempercayai klaim kepemimpinan klien secara pasif. Akibatnya, jeda eksekusi leader lama dapat merusak data yang baru saja ditulis oleh leader pengganti. Setelah menerapkan arsitektur fencing, penyimpanan bersama bertindak sebagai filter aktif yang menjamin data tidak akan pernah tertimpa oleh mutasi yang berasal dari generasi kepemimpinan masa lalu.

Before: passive storage vulnerable to split-brain. After: active fencing guarantees immutable epochs.

Test 1: Akuisisi, Renewal, dan Kedaluwarsa Lease

TestCoordinatorLeaseAcquisitionAndRenewal membuktikan siklus hidup lease di coordinator.

```
func TestCoordinatorLeaseAcquisitionAndRenewal(t *testing.T) {
    coord := coordinator.New()
    ttl := 200 * time.Millisecond
    lease1, err := coord.Acquire("leader-lock", "node-1", ttl)
    // Assert: lease1.FencingToken == 1
    _, err = coord.Acquire("leader-lock", "node-2", ttl)
    // Assert: errors.Is(err, coordinator.ErrLeaseHeld)
    time.Sleep(250 * time.Millisecond) // Lampaui TTL 200ms
    lease2, err := coord.Acquire("leader-lock", "node-2", ttl)
    // Assert: lease2.FencingToken == 2
}
```

Unit test pertama membuktikan kebenaran siklus hidup lease pada modul coordinator. Node-1 berhasil memperoleh lease pertama dengan fencing token bernilai 1. Selama lease masih aktif, permintaan akuisisi dari Node-2 langsung ditolak dengan error ErrLeaseHeld. Operasi pembaruan berkala (Renew) terbukti memperpanjang masa berlaku lease tanpa menaikkan nilai token. Setelah dilakukan sleep selama 250 milidetik yang melampaui batas TTL 200 milidetik, lease kedaluwarsa secara otomatis dan Node-2 berhasil mengakuisisi lease dengan token 2.

VERIFIKASI INVARIAN

Mutual Exclusion: Node-2 ditolak ErrLeaseHeld selama lease Node-1 masih dalam masa aktif.
Token Stability: Memanggil Renew() memperpanjang masa aktif tanpa mengubah nilai token (tetap 1).
Monotonic Increment: Setelah TTL kedaluwarsa, akuisisi oleh Node-2 menghasilkan token 2.

Lease renewal preserves the generation token; lease expiration monotonically advances it.

tests/election_test.go:14–56

Test 2: Storage Menolak Token Usang & Duplikat

TestFencedStorageRejectsStaleTokens memverifikasi syarat strict inequality incoming > lastSeen.

```
func TestFencedStorageRejectsStaleTokens(t *testing.T) {
    store := storage.New()
    _ = store.Write("node-1", 10, "state-1") // Sukses: 10 > 0
    _ = store.Write("node-2", 11, "state-2") // Sukses: 11 > 10
    err1 := store.Write("node-1", 10, "stale-state")
    // Assert: errors.Is(err1, storage.ErrStaleFencingToken)
    err2 := store.Write("node-2", 11, "duplicate-token")
    // Assert: errors.Is(err2, storage.ErrStaleFencingToken)
    // Assert: len(store.History()) == 2 (mutasi ilegal tidak dicatat)
}
```

ATURAN STRICT INEQUALITY

Token Mundur Ditolak: Token 10 ditolak setelah storage melihat token 11 (10 < 11).
Token Sama Ditolak: Token 11 duplikat ditolak karena tidak memenuhi syarat strict greater (11 == 11).
Integritas Riwayat: Penyimpanan bersama hanya berisi tepat 2 record legal.

Unit test kedua menguji ketatnya aturan validasi pada lapisan FencedStorage. Penyimpanan pertama kali menerima token 10, kemudian menerima token 11. Saat Node-1 mencoba menulis kembali dengan token 10, operasi ditolak karena 10 lebih kecil dari 11. Begitu pula saat Node-2 mencoba menulis ulang dengan token 11 yang identik, storage menolaknya karena aturan mensyaratkan token harus strictly lebih besar. Hasil pemeriksaan History() membuktikan riwayat penyimpanan hanya berisi tepat dua data yang sah dan menolak semua mutasi ilegal.

Strict inequality incomingToken > lastSeenToken closes both stale replay and duplicate mutation vulnerabilities.

Test 3: Failover Otomatis & Pertahanan Split-Brain

TestLeaderElectionFailoverAndSplitBrainDefense membuktikan skenario GC pause end-to-end.

```
// 1. Leader-1 terpilih dan menulis sukses dengan token1
leader1.SimulatePause(250 * time.Millisecond) // Lebih lama dari TTL 150ms
time.Sleep(200 * time.Millisecond)
// 2. Leader-2 mengambil alih kepemimpinan dengan token2 > token1
err = leader2.PerformFencedWrite("leader-2-write") // SUKSES
// 3. Leader-1 bangun dan mencoba menulis dengan token1 usang
err = store.Write(leader1.ID(), token1, "stale-write-after-gc-pause")
// Assert: errors.Is(err, storage.ErrStaleFencingToken)
```

FAKTA PERILAKU SISTEM

Leader 1 dibekukan selama 250ms melalui SimulatePause yang menghentikan loop renewal.
Kandidat kedua mendeteksi lease habis dan dipromosikan dengan token lebih tinggi.
Saat Leader 1 bangun, percobaan tulisnya digagalkan oleh pagar validasi storage.

Test ketiga mendemonstrasikan skenario inti split-brain yang dipecahkan oleh lab ini. Node leader pertama disimulasikan mengalami stop-the-world pause selama 250 milidetik, melampaui batas lease TTL 150 milidetik. Kandidat kedua mendeteksi kekosongan lease, mengambil alih kepemimpinan, dan sukses menulis dengan token 2. Ketika leader pertama bangun dan mencoba menulis ke storage menggunakan token 1 yang dipegangnya, operasi ditolak mentah-mentah dengan ErrStaleFencingToken. Tes ini mengonfirmasi pertahanan split-brain bekerja sempurna.

Simulated pause triggers clean failover; resumed stale leader is intercepted at storage boundary.

tests/election_test.go:87-154

Studi Kasus Industri: Mekanisme Fencing Nyata

Bagaimana sistem terdistribusi kelas dunia mengimplementasikan ordering tokens.

SISTEM KOORDINASI MODERN

Poin 1

etcd v3: Menggunakan Lease TTL dipadukan dengan Revision Raft 64-bit sebagai fencing token bawaan.

SISTEM KOORDINASI MODERN

Poin 2

Apache ZooKeeper: Menghasilkan Ephemeral Sequential Znodes; nomor urut zxid bertindak sebagai token.

SISTEM KOORDINASI MODERN

Poin 3

Consul: Mengandalkan internal Raft sessions dengan Session TTL dan periodic health checks.

DEBAT REDIS REDLOCK (KLEPPMANN VS ANTIREZ)

Poin 1

Martin Kleppmann (2016): Redlock tidak menghasilkan fencing token monotonik; rentan GC pause & NTP jump.

DEBAT REDIS REDLOCK (KLEPPMANN VS ANTIREZ)

Poin 2

Salvatore Sanfilippo (antirez): Redlock dirancang untuk timed lock praktis, bukan linearizable consensus.

DEBAT REDIS REDLOCK (KLEPPMANN VS ANTIREZ)

Poin 3

Konsensus Industri: Jika resource tidak mendukung fencing check, Redlock tidak menjamin mutual exclusion mutlak.

Riset sistem terdistribusi menunjukkan kesepakatan luas tentang perlunya fencing tokens. etcd memanfaatkan nomor Raft Revision 64-bit yang selalu bertambah sebagai fencing token alami untuk setiap transaksi mutasi. Apache ZooKeeper menggunakan zxid dan urutan pembuatan ephemeral znode. Debat bersejarah antara Martin Kleppmann dan antirez mengenai algoritma Redis Redlock menegaskan prinsip utama: kunci terdistribusi tanpa fencing token monotonik hanya cukup untuk efisiensi sistem, bukan untuk keamanan data yang membutuhkan korektivitas mutlak.

Without monotonic fencing tokens at the storage boundary, distributed locks provide efficiency, not safety.

Mengapa Lease Wajib Menggunakan Monotonic Clock?

Bahaya lompatan wall-clock akibat penyesuaian NTP terhadap perhitungan kedaluwarsa lease.

ANOMALI WALL-CLOCK (REAL-TIME)

Poin 1

Sinkronisasi waktu jaringan (NTP step) dapat memajukan atau memundurkan wall-clock sistem.

ANOMALI WALL-CLOCK (REAL-TIME)

Poin 2

Jika jam melompat mundur: Lease dianggap aktif selamanya, mematikan kemampuan failover otomatis.

ANOMALI WALL-CLOCK (REAL-TIME)

Poin 3

Jika jam melompat maju: Lease dianggap kedaluwarsa prematur saat leader masih sehat.

GARANSI MONOTONIC CLOCK DI GO

Poin 1

time.Now() di Go membaca monotonic clock hardware secara otomatis untuk pengukuran durasi.

GARANSI MONOTONIC CLOCK DI GO

Poin 2

time.Since(start) dan now.Before(expiresAt) kebal terhadap manipulasi penyesuaian jam sistem.

GARANSI MONOTONIC CLOCK DI GO

Poin 3

Coordinator dan Candidate lab ini murni menggunakan durasi monotonik untuk semua logika TTL.

Penggunaan tipe jam yang tepat merupakan detail implementasi yang membedakan sistem amatir dari sistem kelas produksi. Wall-clock (jam dinding) dirancang untuk mencatat waktu kalender manusia dan dapat melompat mundur atau maju secara drastis saat terjadi penyesuaian NTP. Jika perhitungan lease bergantung pada wall-clock, lompatan waktu dapat memicu kegagalan fatal: failover palsu atau lease yang terkunci abadi. Go standard library mengatasi ini dengan menyematkan pembacaan monotonic hardware clock pada fungsi time.Now(), menjamin perhitungan durasi TTL selalu linier.

Wall-clocks tell human time and jump; monotonic clocks measure duration and never step backwards.

Checklist Implementasi Leader Election & Fencing

9 poin verifikasi wajib sebelum menggelar sistem koordinasi ke lingkungan produksi.

- ✓ [ ] Parameter TTL lease dikalibrasi berdasarkan observasi empiris p99 GC pause dan RTT jaringan.
- ✓ [ ] Loop renewal diatur dengan frekuensi aman (renewRate <= TTL / 3) dengan backoff terukur.
- ✓ [ ] Semua perhitungan durasi lease dan renewal murni menggunakan monotonic clock.
- ✓ [ ] Coordinator menghasilkan revision counter yang strictly monotonically increasing di setiap lease baru.
- ✓ [ ] Pembaharuan lease (renew) oleh pemegang yang sama mempertahankan fencing token yang ada.
- ✓ [ ] Shared resource (database/storage) mengimplementasikan check-and-set atomik (incoming > lastSeen).
- ✓ [ ] Permintaan tulis dengan token <= lastSeenToken ditolak dengan error eksplisit yang dapat dipantau.
- ✓ [ ] Metric alerting dipasang untuk mendeteksi lonjakan kejadian penolakan token basi (stale write spike).

Calibrate timeouts empirically, increment tokens monotonically, and enforce boundaries atomically.

- ✓ [ ] Sistem aplikasi memahami batas antara timed lock untuk efisiensi vs linearizable fencing untuk keamanan.

Anti-Pattern: Kesalahan Desain yang Harus Dihindari

Kekeliruan arsitektur yang melumpuhkan jaminan keselamatan koordinasi terdistribusi.

ANTI-PATTERN UMUM

Fencing di Klien: Melakukan validasi token di memori klien sebelum memanggil storage (rentan TOCTOU).
Menerima Token Sama: Menggunakan syarat token >= lastSeen membuka celah eksekusi mutasi ganda.
Token Non-Monotonik: Memakai UUID acak atau timestamp lokal yang tidak memiliki garansi urutan kausal.
Wall-Clock TTL: Menggunakan jam dinding sistem rentan manipulasi penyesuaian NTP clock skew.

CARA REPRODUKSI GAGAL DI LAB

1. Ubah internal/storage/storage.go baris 37 menjadi token < s.lastSeenToken (izinkan token sama).
2. Jalankan go test -v ./tests: TestFencedStorageRejectsStaleTokens langsung gagal pada duplicate check.
3. Hapus validasi storage sama sekali: stale write dari Node-A akan menimpa histori sah milik Node-B!

Mengabaikan prinsip dasar koordinasi dapat berakibat fatal bagi integritas data perusahaan. Anti-pattern yang paling sering dijumpai adalah memvalidasi status kepemimpinan di sisi klien aplikasi alih-alih di gerbang penyimpanan, membuka celah TOCTOU yang lebar. Mengizinkan token yang bernilai sama (menggunakan operator lebih besar atau sama dengan) juga merusak keunikan generasi pemimpin. Untuk membuktikannya di lab, kita cukup melonggarkan validasi di storage.go dan melihat unit test langsung mendeteksi pelanggaran keamanan.

Client-side checks are advisory; only server-side resource fencing guarantees mutual exclusion.

Verifikasi Mandiri: Uji & Jalankan Lab

Reproduksi seluruh bukti teknis, benchmark race detector, dan demo interaktif di terminal Anda.

```
cd labs/30-leader-election
go test -v ./...
go test -race ./...
go run ./cmd/demo
```

RINGKASAN METRIK LAB

Test Suite: 4 test cases lulus 100% tanpa kegagalan (1.17s total runtime).
Race Detector: Bersih dari thread contention dan data races (2.01s pass).
Dependencies: Murni Go Standard Library (sync, time, context, errors) tanpa dependensi pihak ketiga.

Lab ini dirancang agar dapat diverifikasi secara mandiri dalam hitungan detik. Seluruh implementasi dibangun murni menggunakan standard library Go tanpa memerlukan server eksternal seperti Redis atau etcd. Perintah go test -v mengeksekusi empat pengujian komprehensif mulai dari siklus hidup lease hingga pertahanan split-brain. Perintah go test -race memvalidasi keamanan konkurensi di bawah beban goroutine intensif, dan go run demo menampilkan simulasi visual interaktif penolakan mutasi usang.

Zero-dependency standard library code enables instant, deterministic peer verification.

Prinsip Arsitektur: Desain untuk Asinkronitas

Pelajaran abadi dalam merancang sistem terdistribusi yang tahan terhadap kegagalan parsial.

1. LEASE MEMBERIKAN EFISIENSI, BUKAN SAFETY

Lease berbatas waktu sangat baik untuk mencegah duplikasi pekerjaan komputasi.
Namun, lease tidak pernah cukup untuk menjamin integritas penulisan pada shared storage non-idempoten.

2. RESOURCE ADALAH HAKIM TERTINGGI

Klien yang mengalami pause tidak pernah tahu bahwa dirinya sudah kedaluwarsa.
Hanya shared resource yang memiliki perspektif global untuk menolak instruksi usang melalui fencing.

3. URUTAN GENERASI MENGALAHKAN ASUMSI WAKTU

Jangan pernah bergantung pada asumsi bahwa proses Anda akan selesai sebelum waktu T habis.
Gunakan nomor urut generasi (monotonic epoch) yang dapat divalidasi secara deterministik.

Tiga prinsip arsitektur ini berlaku universal di luar implementasi lab Go ini. Pertama, pahami bahwa lease berbasis waktu hanya memberikan timed mutual exclusion yang cocok untuk optimalisasi efisiensi, bukan keselamatan mutlak. Kedua, tempatkan gerbang keamanan di lapisan resource bersama karena proses klien yang membeku tidak memiliki kesadaran akan berlalunya waktu. Ketiga, gantikan asumsi waktu dengan urutan generasi eksplisit; nomor token yang monotonik naik memberikan jaminan kausalitas yang kebal terhadap jeda sistem terdistribusi.

Time is subjective in distributed systems; monotonic order at the resource is absolute truth.

Otoritas Terdistribusi Terjaga, Split-Brain Dicegah.

Lease TTL grants temporal leadership; monotonic fencing tokens protect permanent storage.

Software Engineering Lab 30 - Leader Election dengan Fencing Tokens

internal/coordinator/coordinator.go — Linearizable lease & monotonic revision issuer
internal/candidate/candidate.go — Campaign runner, heartbeat renewal, & pause simulation
internal/storage/storage.go — Fenced storage gate with atomic check-and-set
tests/election_test.go — 4 comprehensive suites (concurrency, failover, rejection)
cmd/demo/main.go — Complete interactive lifecycle demonstration

REPOSITORY

`github.com/lukman-ss/software-engineering-lab/tree/main/labs/30-leader-election`

Lukman / lukman-ss

internal/coordinator/coordinator.go — Linearizable lease & monotonic revision issuer
internal/candidate/candidate.go — Campaign runner, heartbeat renewal, & pause simulation
internal/storage/storage.go — Fenced storage gate with atomic check-and-set
tests/election_test.go — 4 comprehensive suites (concurrency, failover, rejection)
cmd/demo/main.go — Complete interactive lifecycle demonstration
Commit SHA: ba4b6216
Unit Tests: 4/4 PASS (100% clean)
Race Detector: PASS (zero data race)
Repositori: github.com/lukman-ss/software-engineering-lab/tree/main/labs/30-leader-election